CQKUD: Detailed Mathematical Implementation

Equations, circuits, numerical choices, and code correspondence

Repository implementation guide

15 October 2026

Abstract

This document derives the algorithm implemented in `cqkud.py` and explains how the H_4 example obtains its energies and overlap-matrix condition numbers. The implementation uses the *unscaled* sine action $X_\epsilon = \sin(\epsilon \bar{H})$, with $X_0 = 0$. A related OCQKU specification uses $\sin(\epsilon \bar{H})/\epsilon$ and sets its zero-parameter limit to $\bar{H}$; those formulas must not be substituted into this code without changing its moment and recurrence conventions. The main run combines exact dense matrix operations, PennyLane measurement circuits, Chebyshev coefficient updates, and projection of the physical Hamiltonian.

Contents

1 Inputs, units, and Hamiltonian scaling **2**

2 The sine working operator **2**

3 What the two moment circuits measure **2**

4 Lanczos recurrence derived from the moments **3**

5 Chebyshev representation and coefficient update **3**

5.1 Clenshaw evaluation of a state 4

6 Projected Hamiltonian, overlap, and physical energies **4**

6.1 The condition number shown by the H4 plot 5

7 Complete implementation sequence **5**

8 Square H4 example **6**

9 Numerical accuracy and interpretation **9**

A Optional collected-expression verification **9**

1 Inputs, units, and Hamiltonian scaling

The physical qubit Hamiltonian is a Hermitian Pauli expansion

$$H = \sum_{\ell=1}^{L} h_\ell P_\ell, \qquad P_\ell^\dagger = P_\ell, \qquad h_\ell \in \mathbb{R}. \tag{1}$$

The code uses the coefficient one-norm

$$\Lambda = \sum_{\ell=1}^{L} |h_\ell| > 0, \qquad \bar{H} = \frac{H}{\Lambda}. \tag{2}$$

Because every Pauli string has operator norm one, $\|\bar{H}\|_2 \le 1$. Thus the spectrum of $\bar{H}$ lies in $[-1, 1]$, and a dimensionless projected energy $\bar{E}$ becomes $E = \Lambda \bar{E}$ in the input Hamiltonian's units (Hartree for the molecular example).

The normalized reference $|\Phi_0\rangle$ is the Hartree–Fock computational-basis state. For a fresh molecule, `build_molecular_problem` constructs a PennyLane molecular Hamiltonian, its Pauli coefficients, the scale Λ, and this reference state. The H_4 example normally loads a saved Hamiltonian and reference so that different PennyLane versions use the same input. The exact fixed-spin-sector diagonalization in the example supplies a benchmark only; it is not fed into the CQKUD recurrence.

$$(H, |\Phi_0\rangle, \epsilon, K_{\max}) \longrightarrow \bar{H} \longrightarrow X_\epsilon \longrightarrow \{|q_n\rangle, \alpha_n, \beta_n\} \longrightarrow (M, S) \longrightarrow E_{\mathrm{Ritz}}.$$

2 The sine working operator

For real ϵ, the code defines

$$U_\epsilon = i \exp(-i\epsilon \bar{H}), \qquad X_\epsilon = \frac{U_\epsilon + U_\epsilon^\dagger}{2} = \sin(\epsilon \bar{H}). \tag{3}$$

The equality follows from $ie^{-iA} = \sin A + i \cos A$ for Hermitian A. The phase factor i is essential: the Hermitian part of $e^{-i\epsilon\bar{H}}$ alone is $\cos(\epsilon\bar{H})$. X_ϵ is Hermitian and $\|X_\epsilon\|_2 \le 1$. At $\epsilon = 0$, $U_0 = iI$ and $X_0 = 0$; the Krylov space therefore consists only of the reference state.

Relation to the other OCQKU convention. The local OCQKU source document defines $X_\epsilon^{\mathrm{source}} = \sin(\epsilon\bar{H})/\epsilon$ for nonzero ϵ, with $X_0^{\mathrm{source}} = \bar{H}$. Here, for nonzero ϵ, $X_\epsilon = \epsilon X_\epsilon^{\mathrm{source}}$. The present code's α_n, β_n, and second moments are moments of X_ϵ, not of $X_\epsilon^{\mathrm{source}}$. In particular, there is no factor $1/\epsilon$ in Eq. (3).

The implementation obtains X_ϵ by diagonalizing the dense matrix $\bar{H} = V \,\mathrm{diag}(e_j) V^\dagger$ and evaluating

$$X_\epsilon = V \,\mathrm{diag}\big(\sin(\epsilon e_j)\big) V^\dagger. \tag{4}$$

It builds the phase-shifted evolution matrices from the same eigenvectors, and checks that their Hermitian part agrees with Eq. (4).

3 What the two moment circuits measure

Suppose $|q_n\rangle$ is the current normalized state. The two quantities needed for the recurrence are

$$\alpha_n = \langle q_n | X_\epsilon | q_n \rangle, \qquad \mu_{2,n} = \langle q_n | X_\epsilon^2 | q_n \rangle. \tag{5}$$

Both are real, and $\mu_{2,n} \geq 0$. In the usual parameter range, a Hadamard test with controlled U_ϵ gives an ancilla probability difference

$$p_0 - p_1 = \mathrm{Re}\langle q_n | U_\epsilon | q_n \rangle = \alpha_n. \tag{6}$$

For the second moment, $U_\epsilon U_\epsilon^\dagger = I$ implies

$$X_\epsilon^2 = \frac{U_\epsilon^2 + (U_\epsilon^\dagger)^2 + 2I}{4}, \tag{7}$$

$$\mu_{2,n} = \frac{1 + \mathrm{Re}\langle q_n | U_\epsilon^2 | q_n \rangle}{2}. \tag{8}$$

A second Hadamard test therefore uses controlled U_ϵ^2. The code forms the difference of its two ancilla probabilities to obtain the real overlap in Eq. (8).

Each circuit prepares $|q_n\rangle$ with a dense state-preparation unitary whose first column is the target state. This is an exact small-system realization: the polynomial state is first assembled classically, then loaded into the circuit. The code does not claim an efficient gate decomposition of the polynomial.

For $0 < |\epsilon| < 10^{-3}$, subtraction of almost equal ancilla probabilities can obscure $\mu_{2,n} = O(\epsilon^2)$. The code instead measures the Hermitian observables X_ϵ and X_ϵ^2 directly in this regime. At $\epsilon = 0$, it returns $\alpha_n = \mu_{2,n} = 0$ without these circuits.

4 Lanczos recurrence derived from the moments

Set

$$|q_{-1}\rangle = 0, \quad \beta_0 = 0, \quad |q_0\rangle = |\Phi_0\rangle. \tag{9}$$

In exact arithmetic, the Hermitian Lanczos relation is

$$X_\epsilon |q_n\rangle = \beta_{n+1} |q_{n+1}\rangle + \alpha_n |q_n\rangle + \beta_n |q_{n-1}\rangle. \tag{10}$$

Define the residual

$$|r_n\rangle = X_\epsilon |q_n\rangle - \alpha_n |q_n\rangle - \beta_n |q_{n-1}\rangle. \tag{11}$$

Using orthonormality of the earlier Lanczos states and $\langle q_{n-1} | X_\epsilon | q_n \rangle = \beta_n$, its squared norm is

$$\| |r_n\rangle \|^2 = \mu_{2,n} - \alpha_n^2 - \beta_n^2. \tag{12}$$

Consequently,

$$\beta_{n+1} = \sqrt{\max(0, \mu_{2,n} - \alpha_n^2 - \beta_n^2)}, \qquad |q_{n+1}\rangle = \frac{|r_n\rangle}{\beta_{n+1}} \quad (\beta_{n+1} > 0). \tag{13}$$

The code computes β_{n+1} from the *two measured moments* and updates polynomial coefficients to construct the next state; it does not explicitly form the residual vector in the main loop.

The small negative values of the radicand allowed by the configured moment tolerance are clipped to zero. A more negative value raises an error. Iteration stops at $K_{\max}$, when β_{n+1} is below a breakdown threshold, or when a reconstructed state's raw norm violates the exact-mode norm tolerance. A reported zero β can reflect numerical clipping; it is not, by itself, proof of an exact invariant Krylov subspace.

5 Chebyshev representation and coefficient update

The code stores scalar coefficients instead of all previous dense Chebyshev basis vectors. Define a numerically scaled argument

$$s_\epsilon = \begin{cases} |\epsilon|, & 0 < |\epsilon| < 10^{-3}, \\ 1, & \text{otherwise}, \end{cases} \qquad Y_\epsilon = X_\epsilon / s_\epsilon. \tag{14}$$

For the small-ϵ branch, $|\sin(\epsilon e)|/|\epsilon| \le |e| \le 1$, so the eigenvalues of Y_ϵ remain in $[-1, 1]$. The Chebyshev polynomials satisfy

$$T_0(y) = 1, \quad T_1(y) = y, \quad T_{m+1}(y) = 2yT_m(y) - T_{m-1}(y). \tag{15}$$

The state at step n is represented as

$$|q_n\rangle = \sum_{m=0}^{n} c_{n,m} T_m(Y_\epsilon)|\Phi_0\rangle, \qquad c_{0,0} = 1. \tag{16}$$

Coefficients outside their stored range are treated as zero.

For $Y_\epsilon \sum_m c_{n,m} T_m(Y_\epsilon)$, call its Chebyshev coefficients $d_{n,m}$. The multiplication identities are

$$yT_0(y) = T_1(y), \qquad yT_m(y) = \tfrac{1}{2}(T_{m+1}(y) + T_{m-1}(y)) \quad (m \ge 1). \tag{17}$$

The T_0 endpoint is distinct. It gives

$$d_{n,0} = \tfrac{1}{2} c_{n,1}, \qquad d_{n,1} = c_{n,0} + \tfrac{1}{2} c_{n,2}, \qquad d_{n,m} = \tfrac{1}{2}(c_{n,m-1} + c_{n,m+1}) \quad (m \ge 2). \tag{18}$$

Dividing Eq. (10) by s_ϵ then gives the implemented scalar update

$$\boxed{c_{n+1,m} = \frac{d_{n,m} - (\alpha_n/s_\epsilon) c_{n,m} - (\beta_n/s_\epsilon) c_{n-1,m}}{\beta_{n+1}/s_\epsilon}.} \tag{19}$$

This is the recurrence in `_next_chebyshev_coefficients`; the loop handles Eq. (17)'s endpoint separately.

5.1 Clenshaw evaluation of a state

To turn coefficients $c_{n,0}, \ldots, c_{n,N}$ into one vector, the code uses the backward Clenshaw recurrence. Initialize $|b_{N+1}\rangle = |b_{N+2}\rangle = 0$ and calculate, for $m = N, N-1, \ldots, 1$,

$$|b_m\rangle = 2Y_\epsilon |b_{m+1}\rangle - |b_{m+2}\rangle + c_{n,m}|\Phi_0\rangle. \tag{20}$$

The evaluated vector and its norm are

$$|q_n^{\text{raw}}\rangle = Y_\epsilon |b_1\rangle - |b_2\rangle + c_{n,0}|\Phi_0\rangle, \quad \rho_n = \||q_n^{\text{raw}}\rangle\|, \quad |q_n\rangle = |q_n^{\text{raw}}\rangle/\rho_n. \tag{21}$$

For $N = 0$, the raw state is simply $c_{n,0}|\Phi_0\rangle$. After normalization the code divides the stored coefficient list by the same ρ_n. This preserves the representation in Eq. (16); normalization alone does not enforce orthogonality against earlier $|q_j\rangle$.

6 Projected Hamiltonian, overlap, and physical energies

After generating K states, form the measured matrices

$$S_{ij} = \langle q_i | q_j \rangle, \qquad M_{ij} = \langle q_i | \bar{H} | q_j \rangle, \qquad 0 \le i, j < K. \tag{22}$$

For off-diagonal entries, the interference circuit prepares

$$|\Psi_{ij}\rangle = \frac{|0\rangle|q_i\rangle + |1\rangle|q_j\rangle}{\sqrt{2}}. \tag{23}$$

For any Hermitian data observable A,

$$\langle \sigma_x \otimes A \rangle_{\Psi_{ij}} = \mathrm{Re}\langle q_i|A|q_j\rangle, \qquad \langle \sigma_y \otimes A \rangle_{\Psi_{ij}} = \mathrm{Im}\langle q_i|A|q_j\rangle. \tag{24}$$

Using $A = I$ gives S_{ij}; using $A = \bar{H}$ gives M_{ij}. The code sets $S_{ii} = 1$ because each state is normalized, measures M_{ii} directly, fills the other triangle by Hermitian conjugation, and symmetrizes both matrices.

In exact Lanczos arithmetic $S = I$. For `shots=None`, the implementation follows that theoretical assumption and diagonalizes M:

$$M v_j = \bar{E}_j v_j, \qquad E_j = \Lambda \bar{E}_j. \tag{25}$$

The smallest E_j is the reported ground-state estimate. The tridiagonal matrix with diagonal α_n and off-diagonal β_{n+1} represents X_ϵ, not the physical Hamiltonian; its eigenvalues are not the reported physical energies.

For finite-shot runs, the code accounts for a measured nonidentity overlap. Write $S = V\,\mathrm{diag}(s_j)V^\dagger$, retain only eigenvalues above a threshold τ, and define

$$W = V_{\mathrm{keep}}\,\mathrm{diag}(s_j^{-1/2})_{\mathrm{keep}}, \qquad \widetilde{M} = W^\dagger M W. \tag{26}$$

Diagonalizing $\widetilde{M}$ solves the retained-space generalized problem $Mv = \bar{E}Sv$. The code uses

$$\tau = \max\left\{ t_S \max(1, s_{\max}),\ 5\sqrt{K/N_{\mathrm{shots}}} \right\} \tag{27}$$

in finite-shot mode, where t_S is the configured overlap tolerance. Without shots, only the first term is used for diagnostics.

6.1 The condition number shown by the H4 plot

For retained positive eigenvalues of S, the reported quantity is

$$\kappa_{\mathrm{reported}}(S) = \frac{s_{\mathrm{max,retained}}}{s_{\mathrm{min,retained}}}. \tag{28}$$

At full rank this is the usual spectral condition number $\kappa_2(S)$. If modes are discarded, it is the condition number of the retained subspace, not of the original singular matrix. The code also records $\max_{ij} |S_{ij} - \delta_{ij}|$ as an orthogonality error. The H_4 plot displays the actual values in Eq. (28), close to one; it does not multiply $\kappa - 1$ by any display factor.

7 Complete implementation sequence

1. Build or load $H, |\Phi_0\rangle$, compute Λ and $\bar{H}$.
2. Construct U_ϵ, X_ϵ, and the Chebyshev argument Y_ϵ. Set $c_{0,0} = 1$, $\beta_0 = 0$.
3. At step n, evaluate and normalize $|q_n\rangle$ by Eqs. (20)–(21).
4. Measure α_n and $\mu_{2,n}$ by Eqs. (6)–(8), or with direct Hermitian observables for very small nonzero $|\epsilon|$.
5. Compute β_{n+1} from Eq. (12). Stop at breakdown or at the requested dimension.
6. Otherwise, update $c_{n+1,m}$ using Eqs. (18)–(19) and continue. No future state is precomputed.
7. Measure S and M, compute their diagnostics, diagonalize the appropriate projected problem, and multiply energies by Λ.

Mathematical object	Code location
Molecular input, Pauli scale, reference	`build_molecular_problem`, `load_archived_problem`
Sine action and phase-shifted evolution	`QuantumOCQKUD.__init__`
Moment circuits and extraction	`build_alpha_circuit`, `build_second_moment_circuit`, `measure_moments`
Chebyshev update and Clenshaw evaluation	`_next_chebyshev_coefficients`, `_clenshaw_state`
Main Lanczos loop	`QuantumOCQKUD.run`
Interference circuits and projections	`build_transition_circuit`, `_project_subspace`
Rank, condition, and Ritz solve	`_overlap_diagnostics`, `_diagonalize_projection`

Table 1: Direct map from the derivation to the current Python implementation. All methods except the standalone helpers are in `cqkud.py`.

8 Square H4 example

`examples/h4.py` uses a square of side 1.5 Å, STO-3G, four active electrons in four spatial orbitals, and an eight-qubit Hamiltonian. The bundled snapshot has 93 Pauli terms, $\Lambda = 5.717610381693$ Hartree, and $|\Phi_0\rangle = |11110000\rangle$. The balanced $N_\alpha = N_\beta = 2$ sector contains $\binom{4}{2}^2 = 36$ determinants. Its lowest exact eigenvalue is -1.955125015330 Hartree; this is the example's active-space FCI benchmark.

The example runs $\epsilon = 0.5, 0.8, 1.2$ through at most eight Krylov states and writes the energies, α_n, β_{n+1}, and $\kappa(S)$ to `output/output.txt`. It saves the three plots in `output/`. Reproduce them from the repository root with `python -m examples.h4`. The option `-rebuild` creates a new Hamiltonian with the installed PennyLane version rather than using the bundled snapshot.

ϵ	$K = 8$ energy (Hartree)	$K = 8$ $\kappa(S)$
0.5	-1.955125014664	1.00000407
0.8	-1.955125010809	1.00000383
1.2	-1.955125015327	1.00000001

Table 2: Representative values from the bundled `output/output.txt`. The final digits of a near-breakdown step can change with floating-point execution. The condition numbers are raw values, not excess values.

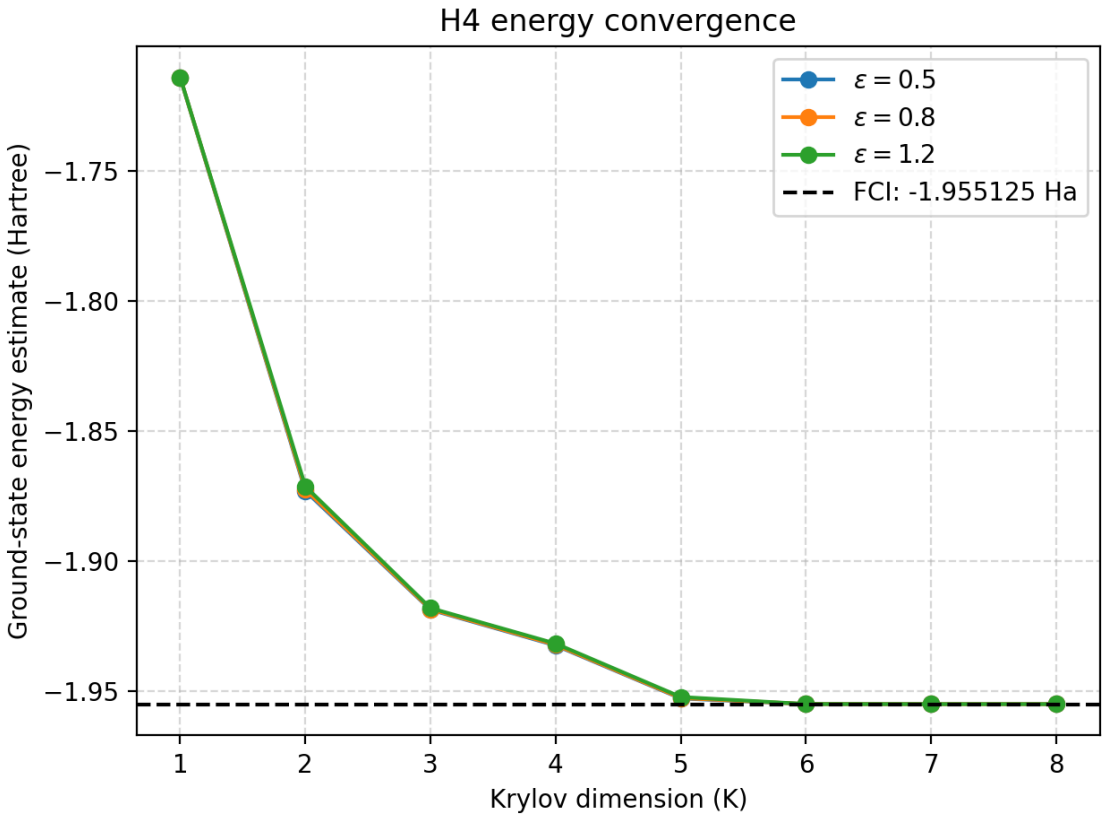


Figure 1: Physical projected energies in the saved H_4 example. The horizontal line is the fixed-sector FCI benchmark.

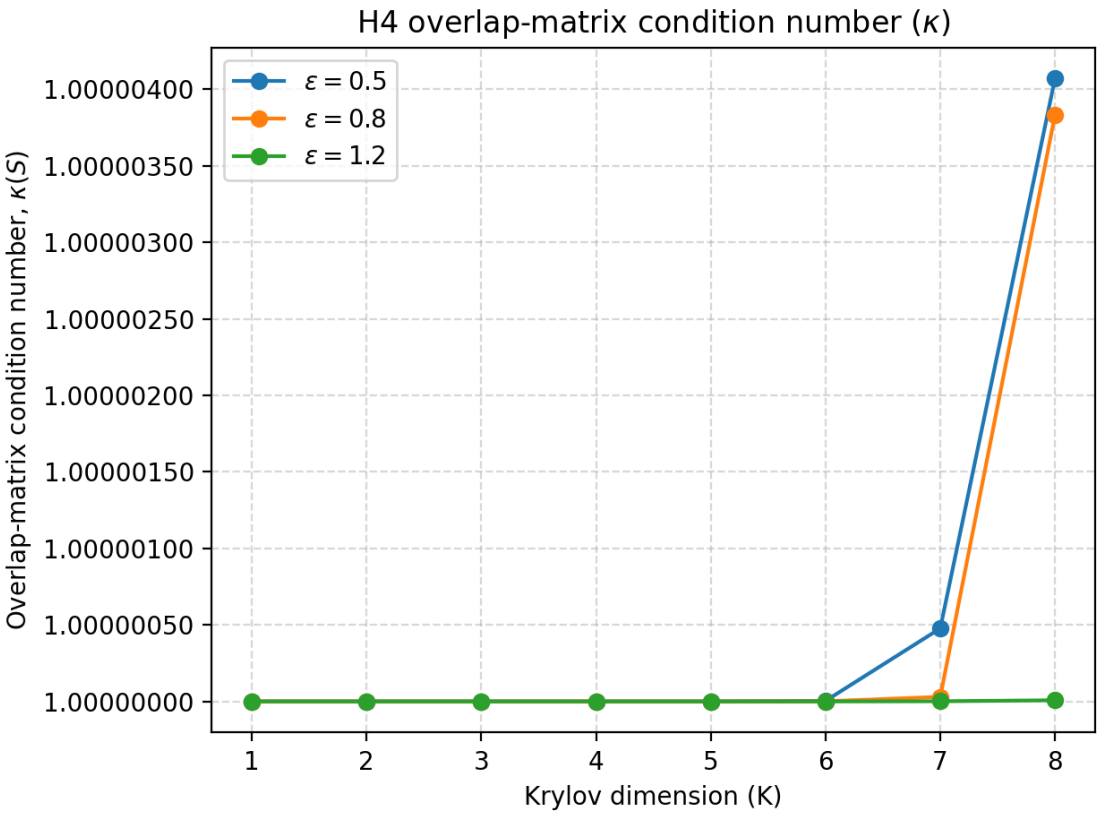


Figure 2: Actual overlap condition number $\kappa(S) = s_{\max}/s_{\min}$, without excess scaling. The apparent late-step rise is small in absolute terms.

9 Numerical accuracy and interpretation

Three-term Lanczos produces orthogonal states in exact arithmetic. This implementation reconstructs later states from Chebyshev coefficient lists in floating-point arithmetic and does not reorthogonalize them. For the H_4 example the later coefficients can become very large while their polynomial sum produces a unit vector; cancellation leaves small overlaps with earlier states. Normalization controls the state length, not those overlaps. Thus $\kappa(S)$ slightly above one is a meaningful diagnostic, but the plotted values near 1.00000 do not indicate a nearly singular basis. Near recurrence breakdown, the final digits of β, energy, and $\kappa(S)$ are sensitive to the floating-point execution path.

The moment identity in Eq. (12) relies on orthogonality. In exact-mode code, Eq. (25) also assumes $S = I$, even when floating-point reconstruction gives S a small deviation from identity. Finite-shot mode instead solves the retained generalized problem. A stricter orthogonal basis would require reorthogonalization, a more stable state recurrence, or higher precision; those are not part of the current `run()` path.

Dense diagonalization, generic dense state preparation, and exact dense time-evolution matrices scale exponentially with qubit count. The implementation is therefore an exact small-system study of the measurement and recurrence formulas, not an efficient large-system quantum circuit construction.

A Optional collected-expression verification

`collected_expressions()` is a separate high-precision reference calculation. It is not called by `run()` and does not generate its circuit measurements. Let the monic Lanczos polynomials in x satisfy

$$p_0(x) = 1, \quad p_1(x) = x - \alpha_0, \quad p_{n+1}(x) = (x - \alpha_n)p_n(x) - \beta_n^2 p_{n-1}(x). \tag{29}$$

With $B_n = \prod_{j=1}^{n} \beta_j$, the ideal normalized state is $|q_n\rangle = p_n(X_\epsilon)|\Phi_0\rangle / B_n$. Write $p_n(x)^2 = \sum_{p=0}^{2n} b_p x^p$. Since $X_\epsilon = (U_\epsilon + U_\epsilon^\dagger)/2$ and U_ϵ is unitary, the coefficient of U_ϵ^k in $p_n(X_\epsilon)^2$, for $0 \le k \le 2n$, is

$$a_k = \sum_{\substack{p \ge k \\ p-k \text{ even}}} b_p \, 2^{-p} \binom{p}{(p-k)/2}. \tag{30}$$

Because H, X_ϵ, and U_ϵ are functions of the same Hermitian matrix and commute,

$$\langle q_n|H|q_n\rangle = \frac{a_0\langle\Phi_0|H|\Phi_0\rangle + 2\sum_{k=1}^{2n} a_k \, \mathrm{Re}\langle\Phi_0|HU_\epsilon^k|\Phi_0\rangle}{B_n^2}. \tag{31}$$

The helper recomputes the spectral recurrence and sine/cosine values with `Decimal` arithmetic, expands the right side, and checks it against the direct spectral expectation. This verifies algebra and cancellation in the collected expression; its input Hamiltonian is still the floating-point matrix supplied to the solver.

Source note

This guide documents the code in `cqkud.py` and `examples/h4.py`. Its sine convention matches the local *OCQKUDSIN Mathematical implementation* reference. The related local *OCQKU Step-by-Step Mathematical Implementation* uses the divided-by-ϵ operator described in Section 2.